Building PES-VCS — A Version Control System

from Scratch

Operating Systems

Orange problem solving(Unit 4)

Name : Nitesh Harsur

SRN : PES1UG24CS584

Section : 4J

Phase 1: Object Storage Foundation Screenshot 1A: Output of `./test_objects ` showing all tests passing

```
nitesh@NITESHHARSUR:~/PES1UG24CS584-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e
5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed022
1bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: find .pes/objects -type f showing the shared directory structure

```
nitesh@NITESHHARSUR:~/PES1UG24CS584-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f4
76
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566f
cf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc
12
```

Phase 2: Tree Objects

Screenshot 2A:Output of `./test_tree ` showing all tests passing

```
nitesh@NITESHHARSUR:~/PES1UG24CS584-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B:

```
nitesh@NITESHHARSUR:~/PES1UG24CS584-pes-vcs$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Phase 3: The Index (Staging area) Screenshot 3A:

```
nitesh@NITESHHARSUR:~/PES1UG24CS584-pes-vcs$ ./pes status
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.c
  untracked:  Makefile
  untracked:  object.c
  untracked:  tree.h
  untracked:  commit.c
  untracked:  test_sequence.sh
  untracked:  test_objects.c
  untracked:  index.c.save.1
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  index.c
  untracked:  a.txt
  untracked:  commit.h
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_tree.c
  untracked:  index.c.save
  untracked:  index.h
```

Screenshot 3B:

```
nitesh@NITESHHARSUR:~/PES1UG24CS584-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
```

Phase 4: Commits and History Screenshot 4A:

```
nitesh@NITESHHARSUR:~/PES1UG24CS584-pes-vcs$ ./pes log
commit 80cd3099b11107b9574cbcd62e6cde2be806eb72922b1694cffd1e10a22fe9c1
Author: student
Date:   1776442793

    first commit
```

Screenshot 4B:

```
nitesh@NITESHHARSUR:~/PES1UG24CS584-pes-vcs$ find .pes/objects -type f
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc53
21
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9eb
b4
.pes/objects/9e/3f55faec50c1e776651d14ecfc249ddc3738a0585b00c8b951e0f4cded4b
f3
```

Screenshot 4C:

```
nitesh@NITESHHARSUR:~/PES1UG24CS584-pes-vcs$ cat .pes/HEAD
cat .pes/refs/heads/main
ref: refs/heads/main
7e2282091df272658248e79dff40fb55f0ca866719adb6e5fea3cce336a0fae6
nitesh@NITESHHARSUR:~/PES1UG24CS584-pes-vcs$
```

Phase 5 and 6: Analysis

Branching and Checkout:

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

Answer: A branch in PES would be implemented as a file inside .pes/refs/heads/ containing the commit hash.

To implement pes checkout <branch>, the following steps are required:

- Update .pes/HEAD to point to the selected branch reference.
- Read the commit hash stored in .pes/refs/heads/<branch>.
- Load the corresponding tree object from the object store.
- Update the working directory to match the tree structure by:
  - Creating files and directories
  - Writing file contents from blob objects

    - Removing files not present in the target branch

This operation is complex because it involves safely modifying the working directory while ensuring no data loss occurs.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store. Answer: To detect a dirty working directory:

- Compare each file in the index with the working directory using:
    - File size
    - Modification time (mtime) Steps:
- If a file in the index has different metadata than the working directory → modified

- If file missing → deleted
- If differences exist and target branch has different version → conflict

Thus, checkout must refuse to proceed to avoid overwriting uncommitted changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits? Answer:

Detached HEAD occurs when .pes/HEAD contains a commit hash instead of a branch reference. In this state:

- New commits are created but not linked to any branch
- These commits can become unreachable (lost)

Recovery:

- Use the commit hash to create a new branch:

pes branch <name> <commit-hash>

- Or manually update .pes/refs/heads/ **Garbage Collection and Space Reclamation**

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit. Answer:

To find unreachable objects:

Algorithm:

1. Start from all branch heads (.pes/refs/heads/*)
2. Traverse commit graph:
   - Visit commits
   - Visit associated tree objects
   - Visit all blobs

3. Mark all visited objects as reachable Data structure:

- Use a **hash set** to store visited hashes Finally:
- Iterate through .pes/objects
- Delete objects not in reachable set Estimation:
- 100,000 commits + trees + blobs ≈ several hundred thousand objects
- All must be traversed

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this? Answer:

Running GC during commit is dangerous due to race conditions:

Example:

- Commit is creating new objects but hasn’t updated references yet
- GC runs simultaneously
- GC does not see new objects as reachable
- GC deletes them
- Commit finishes → references missing objects → corruption Git avoids this by:

- Using locks during operations
- Performing GC only when safe
- Using reference counting and packfiles